

Informe Técnico — CampusLab

Arquitectura Cloud Native segura (Caso 2)

Federación de identidad Microsoft Entra ID + Amazon Cognito, API Gateway, BFF y microservicios en EC2

Williams Rivas

Ingeniería en Informática

DSY1107 — Desarrollo Cloud Native I

Evaluación Parcial (Encargo + Presentación)

Septiembre 2026

ÍNDICE

1. Introducción	3
2. Arquitectura de la solución	3
3. Identidad: Entra ID + Cognito (federación)	4
4. API Gateway y seguridad (JWT, CORS, roles)	5
5. Evidencia del CRUD real (200/400/401/403)	6
6. Despliegue en EC2 con Docker	7
7. Conclusiones y alcance	8

1. INTRODUCCIÓN

Este informe documenta la arquitectura base y segura del sistema **CampusLab**, plataforma de reserva de laboratorios y equipos académicos. La solución integra un **frontend React** con autenticación de **Microsoft (Entra ID)**, identidad federada con **Amazon Cognito**, un **API Gateway** que valida el JWT, un **BFF** con Spring Security y microservicios **Spring Boot** desplegados en **EC2** con **Docker**. Toda la evidencia mostrada es real, obtenida contra los recursos efectivamente aprovisionados en AWS y Azure.

2. ARQUITECTURA DE LA SOLUCIÓN

El flujo seguro es siempre: **JWT** → **API Gateway** → **BFF** → **microservicio de dominio**. El usuario se autentica con Microsoft; Cognito (federado a Entra) emite un access token; el API Gateway valida ese token en el borde y el BFF lo revalida y aplica autorización por rol antes de enrutar a los microservicios **catalog** y **bookings**.

Componente	Tecnología	Recurso real
Frontend	React + Vite + MSAL	http://localhost:5173
Identidad IDaaS	Amazon Cognito (OIDC)	us-east-1_dBOQeniQq
Federación	Microsoft Entra ID	tenant d84b2661 (WRIVAST)
Puerta de entrada	API Gateway (HTTP API)	3wwwncomjkh.execute-api.us-east-1
BFF	Spring Boot + Spring Security	EC2 3.237.97.7:8080
Microservicios	Spring Boot + JPA	Docker (catalog, bookings)

3. IDENTIDAD: ENTRA ID + COGNITO (FEDERACIÓN)

Se registró una aplicación en **Microsoft Entra ID** y se configuró Cognito con un proveedor **OIDC** apuntando a Entra, grupos **Admin/Operador** y un App Client público con **Authorization Code + PKCE**. Al iniciar sesión, el usuario es redirigido al autenticador de Microsoft y Cognito emite un token con los *claims* validados por el backend:

```
{
  "iss": "https://cognito-idp.us-east-1.amazonaws.com/us-east-1_dBOQeniQq",
  "client_id": "6r3af8ucc96okla0ujph9pru4a",
  "token_use": "access",
  "username": "demo.admin",
  "cognito:groups": [ "Admin" ]
}
```

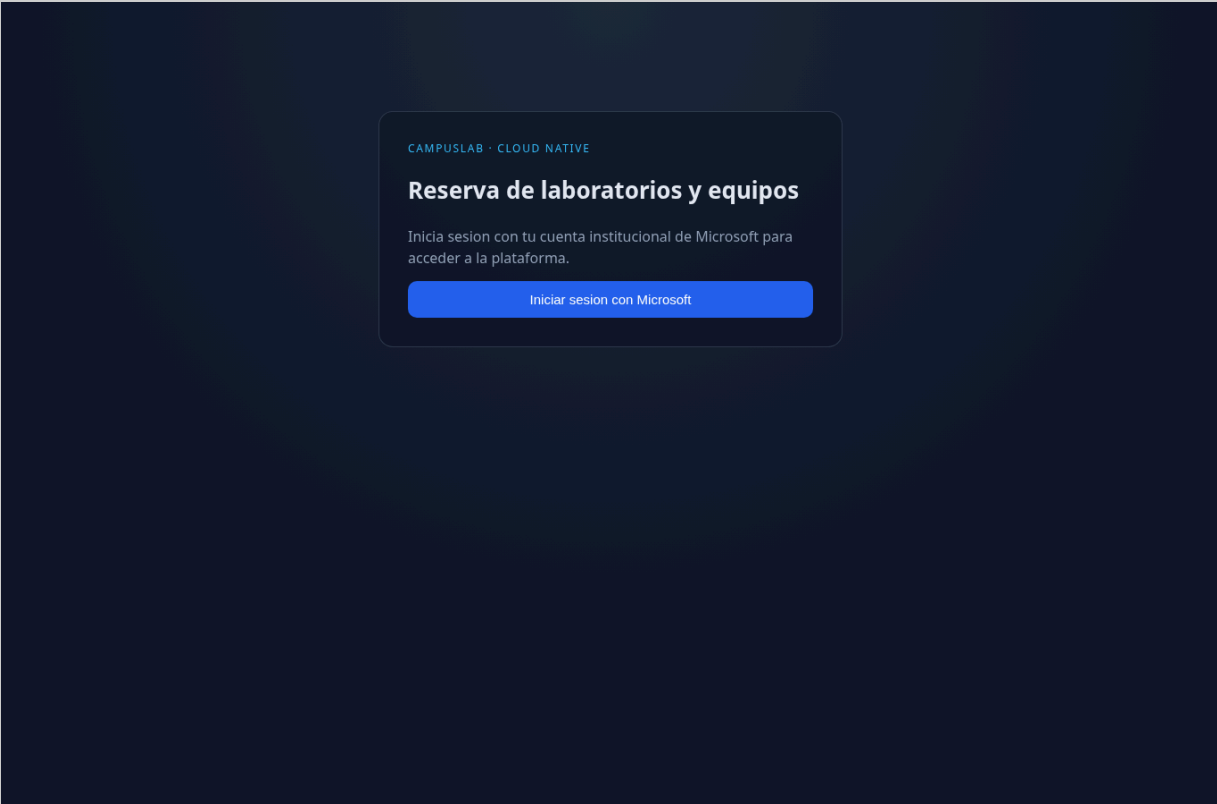


Figura 1. Frontend real: pantalla de inicio de sesión con Microsoft.

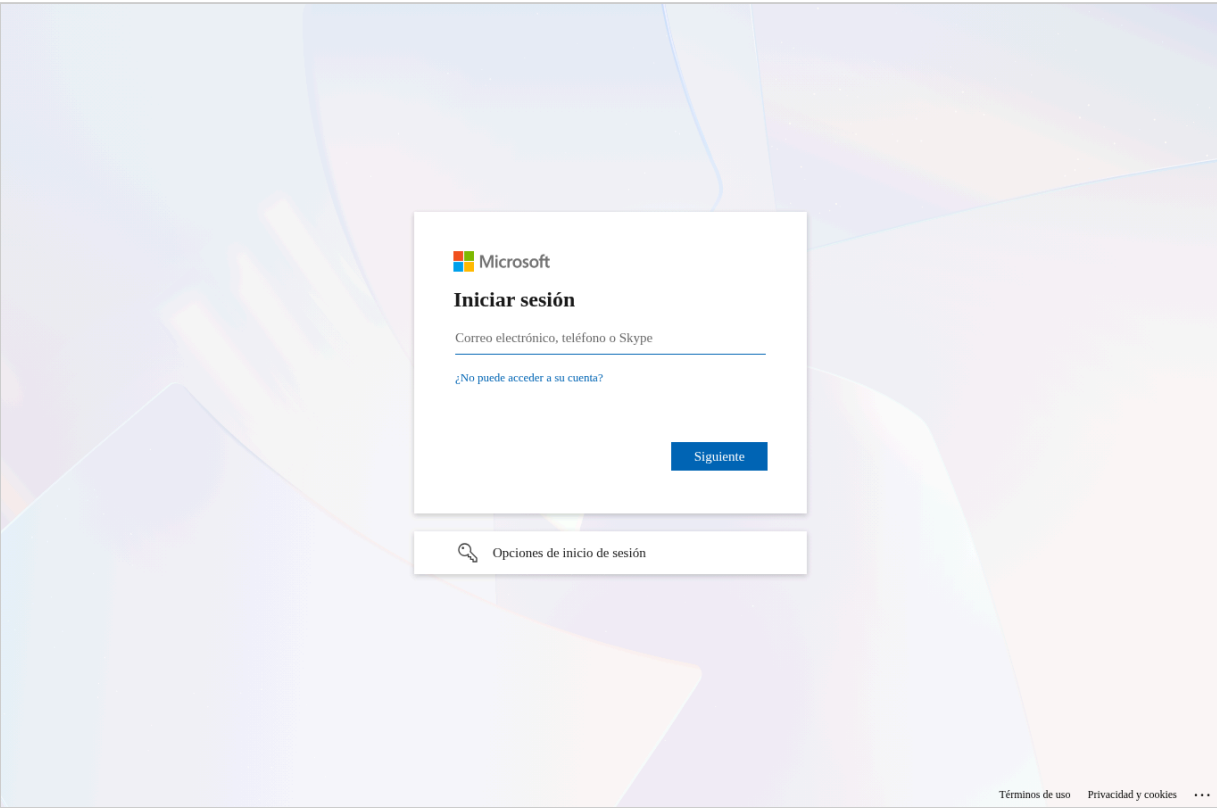


Figura 2. Federación real: Cognito redirige al login de Microsoft (login.microsoftonline.com, tenant WRIVAST).

4. API GATEWAY Y SEGURIDAD (JWT, CORS, ROLES)

El **API Gateway (HTTP API)** expone la ruta ANY / {proxy+} protegida por un **JWT Authorizer** de Cognito (Issuer del User Pool + Audience = client_id) y con **CORS** restringido al origen del frontend. El **BFF** revalida el token (firma, issuer, token_use=access, client_id) y controla el rol por endpoint.

Control	Configuración real
Authorizer	JWT · Issuer: cognito-idp.us-east-1.amazonaws.com/us-east-1_dBOQeniQq
Audience	6r3af8ucc96okla0ujph9pru4a
CORS	Origin http://localhost:5173 · GET,POST,PUT,DELETE,OPTIONS

5. EVIDENCIA DEL CRUD REAL (200/400/401/403)

Batería ejecutada contra el **Invoke URL real** del API Gateway, con tokens reales emitidos por Cognito. Se demuestran los cuatro verbos y los códigos de estado exigidos por la rúbrica:

```
1) GET /api/catalog/resources SIN token
   -> HTTP 401 {"message":"Unauthorized"} (rechazado por el API Gateway)

2) GET /api/catalog/resources (token Admin)
   -> HTTP 200 [{"id":1,"nombre":"Laboratorio de Redes 3","tipo":"LABORATORIO","stock":1,...},
               {"id":3,"nombre":"Osciloscopio Rigol DS1054Z","tipo":"EQUIPO","stock":8,...},
               ...]

3) POST /api/catalog/resources (Admin) {"nombre":"Laboratorio de IoT","tipo":"LABORATORIO",...}
   -> HTTP 201 {"id":6,"nombre":"Laboratorio de IoT","tipo":"LABORATORIO","stock":2,"disponible":true}

4) PUT /api/catalog/resources/1 (Admin)
   -> HTTP 200 {"id":1,"nombre":"Laboratorio de Redes 3","stock":3,...}

5) POST /api/catalog/resources (usuario NO Admin)
   -> HTTP 403 (rechazado por el BFF: el rol no puede usar el endpoint)

6) POST /api/bookings (Admin) {"nombre":"Ana Perez","rut":"12345678-9","fecha":"2026-09-20T10:30:00","productoId":1}
   -> HTTP 201 {"id":3,"nombre":"Ana Perez","rut":"12345678-9","estado":"SOLICITADA",...}

7) POST /api/bookings RUT/nombre invalidos (validacion de esquema)
   -> HTTP 400 {"error":"Bad Request","message":"La validacion de esquema fallo",
               "errores":{"rut":"El rut debe tener formato 12345678-9 (sin puntos)",
                           "nombre":"El nombre es obligatorio"}}

8) PUT /api/bookings/2/status (Admin) {"status":"EN_USO"}
   -> HTTP 200 {"id":2,"nombre":"Bruno Soto","estado":"EN_USO",...}

9) DELETE /api/bookings/1 (Admin)
   -> HTTP 204 (sin contenido)
```

Resultado: 401 (sin token) · 200 (GET) · 201 (POST) · 200 (PUT) · 403 (rol sin permiso) · 400 (validación de esquema) · 204 (DELETE). Todo real, extremo a extremo.

```
CampusLab - CRUD real via API Gateway (consola)

# Sesión de consola - CRUD real via API Gateway (https://3wvnmcomjnhk.execute-api.us-east-1.amazonaws.com)
# TOKEN=<access token Cognito> (admin)

$ curl $B/api/catalog/resources # sin token
{"message":"Unauthorized"}
-> HTTP 401

$ curl -H "Authorization: Bearer $TOKEN" $B/api/catalog/resources
[{"id":1,"nombre":"Laboratorio de Redes 3","tipo":"LABORATORIO","stock":2,"disponible":true},{"id":2,"nombre":"Laboratorio de Electronica 1","tipo":"LABORATORIO","stock":1,"disponible":true},{"id":3,"nombre":"Osciloscopio Rigol DS1054Z","tipo":"EQUIPO","stock":8,"disponible":true}, {"id":4,"nombre":"Kit Arduino Uno","tipo":"EQUIPO","stock":25,"disponible":true}, {"id":6,"nombre":"Laboratorio de IoT","tipo":"LABORATORIO","stock":2,"disponible":true}, {"id":7,"nombre":"Laboratorio de IoT","tipo":"LABORATORIO","stock":1,"disponible":true}]
-> HTTP 200

$ curl -X POST -H "Authorization: Bearer $TOKEN" -d '{...IoT...}' $B/api/catalog/resources
{"id":8,"nombre":"Laboratorio de IoT","tipo":"LABORATORIO","stock":2,"disponible":true}
-> HTTP 201

$ curl -X POST -H "Authorization: Bearer $USER_TOKEN" -d '{...}' $B/api/catalog/resources # rol NO admin
-> HTTP 403

$ curl -X POST -H "Authorization: Bearer $TOKEN" -d '{rut invalido}' $B/api/bookings
{"error":"Bad Request","message":"La validacion de esquema fallo","errores":{"rut":"El rut debe tener formato 12345678-9 (sin puntos)","nombre":"El nombre es obligatorio"},"status":400}
-> HTTP 400

$ curl -X POST -H "Authorization: Bearer $TOKEN" -d '{reserva valida}' $B/api/bookings
{"id":5,"nombre":"Ana Perez","rut":"12345678-9","fecha":"2026-09-20T10:30:00","productoId":1,"estado":"SOLICITADA"}
-> HTTP 201
```

Figura 3. Consola: CRUD real ejecutado contra el API Gateway (401/200/201/403/400/201).

5.1 Evidencia con Postman (newman)

La colección de Postman se ejecutó con **newman** contra el Invoke URL real. Resultado: **13 requests, 8 assertions, 0 fallidas**.

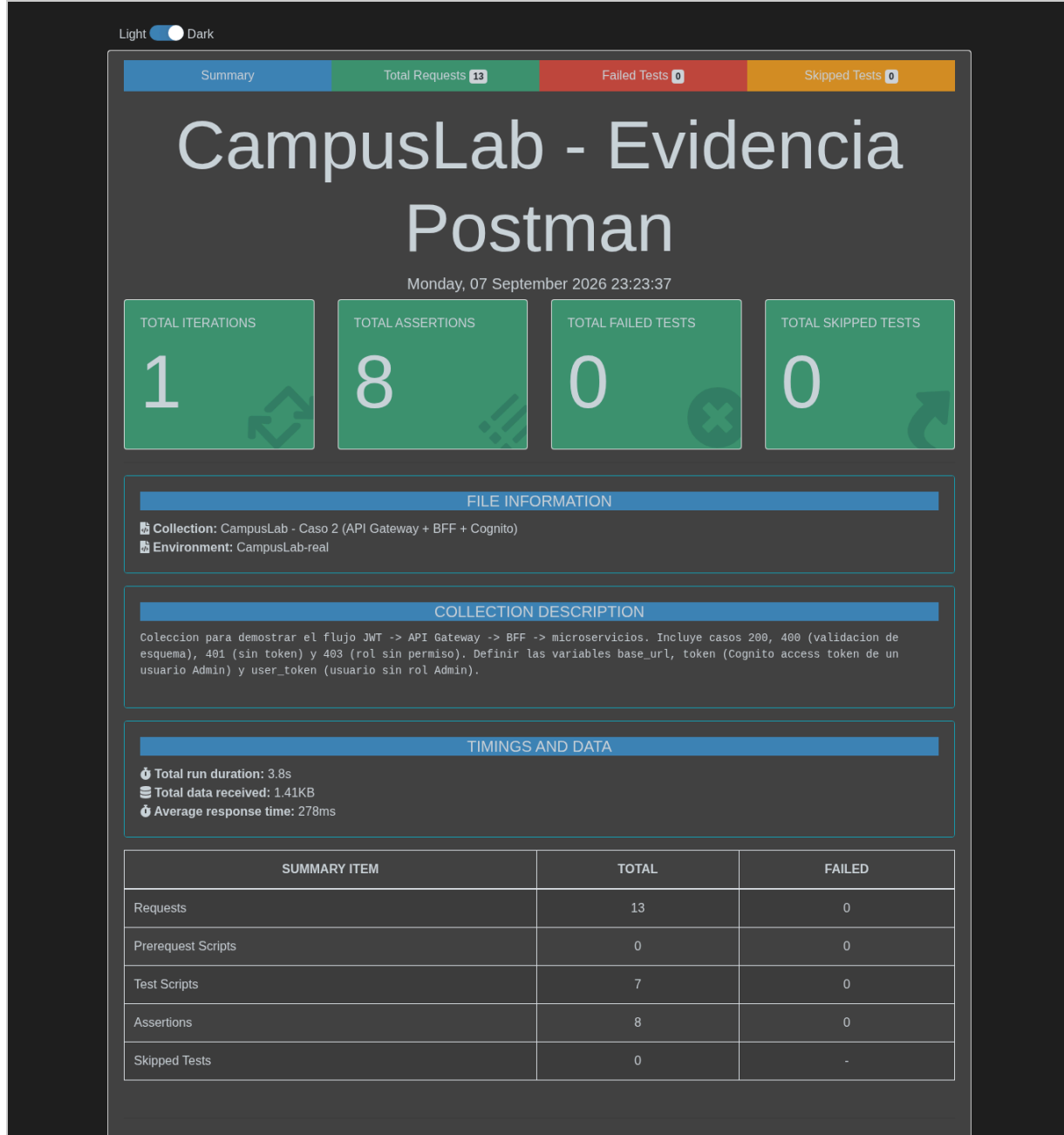


Figura 4. Reporte de Postman/newman: 8 aserciones correctas, 0 fallos.

6. DESPLIEGUE EN EC2 CON DOCKER

Los microservicios **catalog**, **bookings** y el **BFF** se ejecutan en una instancia **EC2 (Amazon Linux 2023)** mediante **Docker Compose**. El Security Group abre solo los puertos necesarios (22, 8080, 8090).

```
$ sudo docker compose ps
NAME                IMAGE                                STATUS      PORTS
campuslab-catalog   eclipse-temurin:17-jre              Up (healthy) 8081
campuslab-bookings   eclipse-temurin:17-jre              Up (healthy) 8082
campuslab-bff        eclipse-temurin:17-jre              Up (healthy) 0.0.0.0:8080->8080

$ curl http://3.237.97.7:8080/actuator/health -> {"status":"UP"}
$ curl http://3.237.97.7:8080/api/catalog/resources (sin token) -> HTTP 401
```

Recurso EC2	Valor
Instancia	i-0448753bab6e8173b (t3.medium, running)
IP pública	3.237.97.7
Contenedores	catalog, bookings, bff

7. CONCLUSIONES Y ALCANCE

Se implementó y desplegó una arquitectura Cloud Native **funcional y segura**: identidad federada a Microsoft, puerta única en API Gateway con validación JWT, BFF con autorización por rol, CRUD con validación de esquema (nombre/rut/fecha) y microservicios en EC2/Docker. La evidencia real confirma los códigos 200/201/204/400/401/403.

Fuera de alcance por decisión del encargo: mensajería Kafka/streaming, KPIs/analytics, auditoría por streaming y AWS Lambda (la integración del API Gateway es HTTP directa al BFF).

Repositorio: github.com/WilliamsRT93/campuslab-caso2